

Special Matrices and Gauss-Seidel

Certain matrices have a particular structure that can be exploited to develop efficient solution schemes. The first part of this chapter is devoted to two such systems: *banded* and *symmetric matrices*. Efficient elimination methods are described for both.

The second part of the chapter turns to an alternative to elimination methods, that is, approximate, iterative methods. The focus is on the *Gauss-Seidel method*, which employs initial guesses and then iterates to obtain refined estimates of the solution. The Gauss-Seidel method is particularly well suited for large numbers of equations. In these cases, elimination methods can be subject to round-off errors. Because the error of the Gauss-Seidel method is controlled by the number of iterations, round-off error is not an issue of concern with this method. However, there are certain instances where the Gauss-Seidel technique will not converge on the correct answer. These and other trade-offs between elimination and iterative methods will be discussed in subsequent pages.

11.1 SPECIAL MATRICES

As mentioned in Box PT3.1, a *banded matrix* is a square matrix that has all elements equal to zero, with the exception of a band centered on the main diagonal. Banded systems are frequently encountered in engineering and scientific practice. For example, they typically occur in the solution of differential equations. In addition, other numerical methods such as cubic splines (Sec. 18.5) involve the solution of banded systems.

The dimensions of a banded system can be quantified by two parameters: the bandwidth BW and the half-bandwidth HBW (Fig. 11.1). These two values are related by $BW = 2HBW + 1$. In general, then, a banded system is one for which $a_{ij} = 0$ if $|i - j| > HBW$.

Although Gauss elimination or conventional LU decomposition can be employed to solve banded equations, they are inefficient, because if pivoting is unnecessary none of the elements outside the band would change from their original values of zero. Thus, unnecessary space and time would be expended on the storage and manipulation of these useless zeros. If it is known beforehand that pivoting is unnecessary, very efficient algorithms can be developed that do not involve the zero elements outside the band. Because many problems involving banded systems do not require pivoting, these alternative algorithms, as described next, are the methods of choice.

(a) Decomposition

```

DOFOR k = 2, n
    ek = ek / fk-1
    fk = fk - ek · gk-1
END DO

```

(b) Forward substitution

```

DOFOR k = 2, n
    rk = rk - ek · rk-1
END DO

```

(c) Back substitution

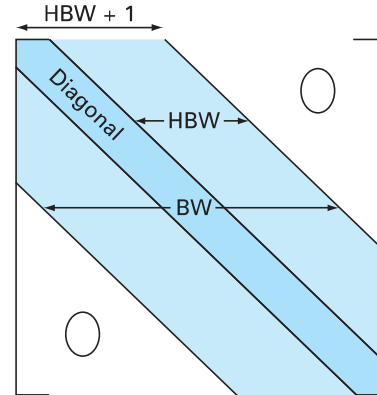
```

xn = rn / fn
DOFOR k = n - 1, 1, -1
    xk = (rk - gk · xk+1) / fk
END DO

```

FIGURE 11.2

Pseudocode to implement the Thomas algorithm, an *LU* decomposition method for tridiagonal systems.

**FIGURE 11.1**

Parameters used to quantify the dimensions of a banded system. BW and HBW designate the bandwidth and the half-bandwidth, respectively.

11.1.1 Tridiagonal Systems

A tridiagonal system—that is, one with a bandwidth of 3—can be expressed generally as

$$\begin{bmatrix} f_1 & g_1 & & & \\ e_2 & f_2 & g_2 & & \\ & e_3 & f_3 & g_3 & \\ & & \ddots & \ddots & \ddots \\ & & & e_{n-1} & f_{n-1} & g_{n-1} \\ & & & & e_n & f_n \end{bmatrix} \begin{Bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \\ x_n \end{Bmatrix} = \begin{Bmatrix} r_1 \\ r_2 \\ r_3 \\ \vdots \\ r_{n-1} \\ r_n \end{Bmatrix} \quad (11.1)$$

Notice that we have changed our notation for the coefficients from *a*'s and *b*'s to *e*'s, *f*'s, *g*'s, and *r*'s. This was done to avoid storing large numbers of useless zeros in the square matrix of *a*'s. This space-saving modification is advantageous because the resulting algorithm requires less computer memory.

Figure 11.2 shows pseudocode for an efficient method, called the *Thomas algorithm*, to solve Eq. (11.1). As with conventional *LU* decomposition, the algorithm consists of three steps: decomposition and forward and back substitution. Thus, all the advantages of *LU* decomposition, such as convenient evaluation of multiple right-hand-side vectors and the matrix inverse, can be accomplished by proper application of this algorithm.

EXAMPLE 11.1**Tridiagonal Solution with the Thomas Algorithm**

Problem Statement. Solve the following tridiagonal system with the Thomas algorithm.

$$\begin{bmatrix} 2.04 & -1 & & \\ -1 & 2.04 & -1 & \\ & -1 & 2.04 & -1 \\ & & -1 & 2.04 \end{bmatrix} \begin{Bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{Bmatrix} = \begin{Bmatrix} 40.8 \\ 0.8 \\ 0.8 \\ 200.8 \end{Bmatrix}$$

Solution. First, the decomposition is implemented as

$$\begin{aligned}e_2 &= -1/2.04 = -0.49 \\f_2 &= 2.04 - (-0.49)(-1) = 1.550 \\e_3 &= -1/1.550 = -0.645 \\f_3 &= 2.04 - (-0.645)(-1) = 1.395 \\e_4 &= -1/1.395 = -0.717 \\f_4 &= 2.04 - (-0.717)(-1) = 1.323\end{aligned}$$

Thus, the matrix has been transformed to

$$\begin{bmatrix} 2.04 & -1 & & \\ -0.49 & 1.550 & -1 & \\ & -0.645 & 1.395 & -1 \\ & & -0.717 & 1.323 \end{bmatrix}$$

and the LU decomposition is

$$[A] = [L][U] = \begin{bmatrix} 1 & & & \\ -0.49 & 1 & & \\ & -0.645 & 1 & \\ & & -0.717 & 1 \end{bmatrix} \begin{bmatrix} 2.04 & -1 & & \\ & 1.550 & -1 & \\ & & 1.395 & -1 \\ & & & 1.323 \end{bmatrix}$$

You can verify that this is correct by multiplying $[L][U]$ to yield $[A]$.

The forward substitution is implemented as

$$\begin{aligned}r_2 &= 0.8 - (-0.49)40.8 = 20.8 \\r_3 &= 0.8 - (-0.645)20.8 = 14.221 \\r_4 &= 200.8 - (-0.717)14.221 = 210.996\end{aligned}$$

Thus, the right-hand-side vector has been modified to

$$\begin{pmatrix} 40.8 \\ 20.8 \\ 14.221 \\ 210.996 \end{pmatrix}$$

which then can be used in conjunction with the $[U]$ matrix to perform back substitution and obtain the solution

$$\begin{aligned}T_4 &= 210.996/1.323 = 159.480 \\T_3 &= [14.221 - (-1)159.48]/1.395 = 124.538 \\T_2 &= [20.800 - (-1)124.538]/1.550 = 93.778 \\T_1 &= [40.800 - (-1)93.778]/2.040 = 65.970\end{aligned}$$

11.1.2 Cholesky Decomposition

Recall from Box PT3.1 that a symmetric matrix is one where $a_{ij} = a_{ji}$ for all i and j . In other words, $[A] = [A]^T$. Such systems occur commonly in both mathematical and

engineering problem contexts. They offer computational advantages because only half the storage is needed and, in most cases, only half the computation time is required for their solution.

One of the most popular approaches involves *Cholesky decomposition*. This algorithm is based on the fact that a symmetric matrix can be decomposed, as in

$$[A] = [L][L]^T \quad (11.2)$$

That is, the resulting triangular factors are the transpose of each other.

The terms of Eq. (11.2) can be multiplied out and set equal to each other. The result can be expressed simply by recurrence relations. For the k th row,

$$l_{ki} = \frac{a_{ki} - \sum_{j=1}^{i-1} l_{ij} l_{kj}}{l_{ii}} \quad \text{for } i = 1, 2, \dots, k-1 \quad (11.3)$$

and

$$l_{kk} = \sqrt{a_{kk} - \sum_{j=1}^{k-1} l_{kj}^2} \quad (11.4)$$

EXAMPLE 11.2

Cholesky Decomposition

Problem Statement. Apply Cholesky decomposition to the symmetric matrix

$$[A] = \begin{bmatrix} 6 & 15 & 55 \\ 15 & 55 & 225 \\ 55 & 225 & 979 \end{bmatrix}$$

Solution. For the first row ($k = 1$), Eq. (11.3) is skipped and Eq. (11.4) is employed to compute

$$l_{11} = \sqrt{a_{11}} = \sqrt{6} = 2.4495$$

For the second row ($k = 2$), Eq. (11.3) gives

$$l_{21} = \frac{a_{21}}{l_{11}} = \frac{15}{2.4495} = 6.1237$$

and Eq. (11.4) yields

$$l_{22} = \sqrt{a_{22} - l_{21}^2} = \sqrt{55 - (6.1237)^2} = 4.1833$$

For the third row ($k = 3$), Eq. (11.3) gives ($i = 1$)

$$l_{31} = \frac{a_{31}}{l_{11}} = \frac{55}{2.4495} = 22.454$$

and ($i = 2$)

$$l_{32} = \frac{a_{32} - l_{21}l_{31}}{l_{22}} = \frac{225 - 6.1237(22.454)}{4.1833} = 20.917$$

```

DOFOR k = 1, n
  DOFOR i = 1, k - 1
    sum = 0.
    DOFOR j = 1, i - 1
      sum = sum + aij · akj
    END DO
    aki = (aki - sum)/aii
  END DO
  sum = 0.
  DOFOR j = 1, k - 1
    sum = sum + aki2
  END DO
  akk = √(akk - sum)
END DO

```

FIGURE 11.3

Pseudocode for Cholesky's LU decomposition algorithm.

and Eq. (11.4) yields

$$l_{33} = \sqrt{a_{33} - l_{31}^2 - l_{32}^2} = \sqrt{979 - (22.454)^2 - (20.917)^2} = 6.1101$$

Thus, the Cholesky decomposition yields

$$[L] = \begin{bmatrix} 2.4495 & & \\ 6.1237 & 4.1833 & \\ 22.454 & 20.917 & 6.1101 \end{bmatrix}$$

The validity of this decomposition can be verified by substituting it and its transpose into Eq. (11.2) to see if their product yields the original matrix $[A]$. This is left for an exercise.

Figure 11.3 presents pseudocode for implementing the Cholesky decomposition algorithm. It should be noted that the algorithm in Fig. 11.3 could result in an execution error if the evaluation of a_{kk} involves taking the square root of a negative number. However, for cases where the matrix is *positive definite*,¹ this will never occur. Because many symmetric matrices dealt with in engineering are, in fact, positive definite, the Cholesky algorithm has wide application. Another benefit of dealing with positive definite symmetric matrices is that pivoting is not required to avoid division by zero. Thus, we can implement the algorithm in Fig. 11.3 without the complication of pivoting.

11.2 GAUSS-SEIDEL

Iterative or approximate methods provide an alternative to the elimination methods described to this point. Such approaches are similar to the techniques we developed to obtain the roots of a single equation in Chap. 6. Those approaches consisted of guessing a value and then using a systematic method to obtain a refined estimate of the root. Because the present part of the book deals with a similar problem—obtaining the values that simultaneously satisfy a set of equations—we might suspect that such approximate methods could be useful in this context.

The *Gauss-Seidel method* is the most commonly used iterative method. Assume that we are given a set of n equations:

$$[A]\{X\} = \{B\}$$

Suppose that for conciseness we limit ourselves to a 3×3 set of equations. If the diagonal elements are all nonzero, the first equation can be solved for x_1 , the second for x_2 , and the third for x_3 to yield

$$x_1 = \frac{b_1 - a_{12}x_2 - a_{13}x_3}{a_{11}} \quad (11.5a)$$

¹A *positive definite matrix* is one for which the product $\{X\}^T [A]\{X\}$ is greater than zero for all nonzero vectors $\{X\}$.

$$x_2 = \frac{b_2 - a_{21}x_1 - a_{23}x_3}{a_{22}} \quad (11.5b)$$

$$x_3 = \frac{b_3 - a_{31}x_1 - a_{32}x_2}{a_{33}} \quad (11.5c)$$

Now, we can start the solution process by choosing guesses for the x 's. A simple way to obtain initial guesses is to assume that they are all zero. These zeros can be substituted into Eq. (11.5a), which can be used to calculate a new value for $x_1 = b_1/a_{11}$. Then, we substitute this new value of x_1 along with the previous guess of zero for x_3 into Eq. (11.5b) to compute a new value for x_2 . The process is repeated for Eq. (11.5c) to calculate a new estimate for x_3 . Then we return to the first equation and repeat the entire procedure until our solution converges closely enough to the true values. Convergence can be checked using the criterion [recall Eq. (3.5)]

$$|\varepsilon_{a,i}| = \left| \frac{x_i^j - x_i^{j-1}}{x_i^j} \right| 100\% < \varepsilon_s \quad (11.6)$$

for all i , where j and $j - 1$ are the present and previous iterations.

EXAMPLE 11.3

Gauss-Seidel Method

Problem Statement. Use the Gauss-Seidel method to obtain the solution of the same system used in Example 10.2:

$$\begin{aligned} 3x_1 - 0.1x_2 - 0.2x_3 &= 7.85 \\ 0.1x_1 + 7x_2 - 0.3x_3 &= -19.3 \\ 0.3x_1 - 0.2x_2 + 10x_3 &= 71.4 \end{aligned}$$

Recall that the true solution is $x_1 = 3$, $x_2 = -2.5$, and $x_3 = 7$.

Solution. First, solve each of the equations for its unknown on the diagonal.

$$x_1 = \frac{7.85 + 0.1x_2 + 0.2x_3}{3} \quad (E11.3.1)$$

$$x_2 = \frac{-19.3 - 0.1x_1 + 0.3x_3}{7} \quad (E11.3.2)$$

$$x_3 = \frac{71.4 - 0.3x_1 + 0.2x_2}{10} \quad (E11.3.3)$$

By assuming that x_2 and x_3 are zero, Eq. (E11.3.1) can be used to compute

$$x_1 = \frac{7.85 + 0 + 0}{3} = 2.616667$$

This value, along with the assumed value of $x_3 = 0$, can be substituted into Eq. (E11.3.2) to calculate

$$x_2 = \frac{-19.3 - 0.1(2.616667) + 0}{7} = -2.794524$$

The first iteration is completed by substituting the calculated values for x_1 and x_2 into Eq. (E11.3.3) to yield

$$x_3 = \frac{71.4 - 0.3(2.616667) + 0.2(-2.794524)}{10} = 7.005610$$

For the second iteration, the same process is repeated to compute

$$x_1 = \frac{7.85 + 0.1(-2.794524) + 0.2(7.005610)}{3} = 2.990557 \quad |\varepsilon_r| = 0.31\%$$

$$x_2 = \frac{-19.3 - 0.1(2.990557) + 0.3(7.005610)}{7} = -2.499625 \quad |\varepsilon_r| = 0.015\%$$

$$x_3 = \frac{71.4 - 0.3(2.990557) + 0.2(-2.499625)}{10} = 7.000291 \quad |\varepsilon_r| = 0.0042\%$$

The method is, therefore, converging on the true solution. Additional iterations could be applied to improve the answers. However, in an actual problem, we would not know the true answer a priori. Consequently, Eq. (11.6) provides a means to estimate the error. For example, for x_1 ,

$$|\varepsilon_{a,1}| = \left| \frac{2.990557 - 2.616667}{2.990557} \right| 100\% = 12.5\%$$

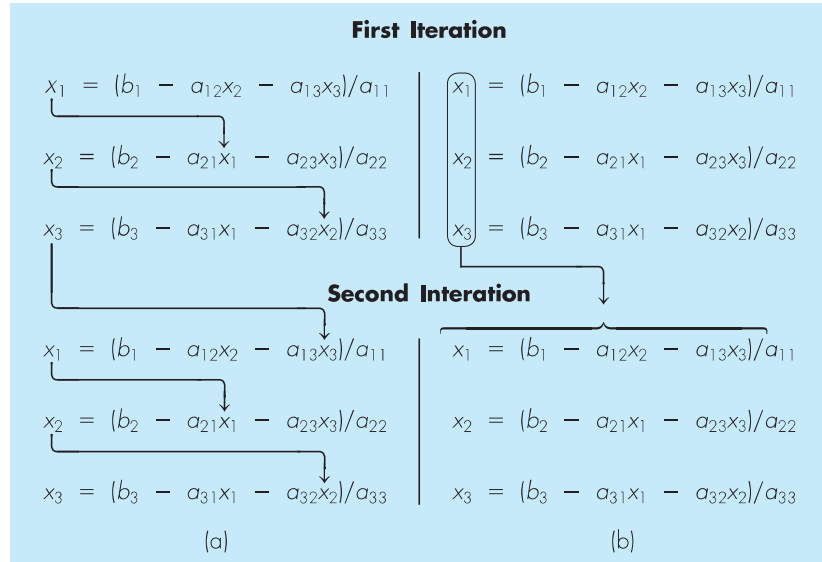
For x_2 and x_3 , the error estimates are $|\varepsilon_{a,2}| = 11.8\%$ and $|\varepsilon_{a,3}| = 0.076\%$. Note that, as was the case when determining roots of a single equation, formulations such as Eq. (11.6) usually provide a conservative appraisal of convergence. Thus, when they are met, they ensure that the result is known to at least the tolerance specified by ε_s .

As each new x value is computed for the Gauss-Seidel method, it is immediately used in the next equation to determine another x value. Thus, if the solution is converging, the best available estimates will be employed. An alternative approach, called *Jacobi iteration*, utilizes a somewhat different tactic. Rather than using the latest available x 's, this technique uses Eq. (11.5) to compute a set of new x 's on the basis of a set of old x 's. Thus, as new values are generated, they are not immediately used but rather are retained for the next iteration.

The difference between the Gauss-Seidel method and Jacobi iteration is depicted in Fig. 11.4. Although there are certain cases where the Jacobi method is useful, Gauss-Seidel's utilization of the best available estimates usually makes it the method of preference.

11.2.1 Convergence Criterion for the Gauss-Seidel Method

Note that the Gauss-Seidel method is similar in spirit to the technique of simple fixed-point iteration that was used in Sec. 6.1 to solve for the roots of a single equation. Recall that simple fixed-point iteration had two fundamental problems: (1) it was sometimes nonconvergent and (2) when it converged, it often did so very slowly. The Gauss-Seidel method can also exhibit these shortcomings.

**FIGURE 11.4**

Graphical depiction of the difference between (a) the Gauss-Seidel and (b) the Jacobi iterative methods for solving simultaneous linear algebraic equations.

Convergence criteria can be developed by recalling from Sec. 6.5.1 that sufficient conditions for convergence of two nonlinear equations, $u(x, y)$ and $v(x, y)$, are

$$\left| \frac{\partial u}{\partial x} \right| + \left| \frac{\partial u}{\partial y} \right| < 1 \quad (11.7a)$$

and

$$\left| \frac{\partial v}{\partial x} \right| + \left| \frac{\partial v}{\partial y} \right| < 1 \quad (11.7b)$$

These criteria also apply to linear equations of the sort we are solving with the Gauss-Seidel method. For example, in the case of two simultaneous equations, the Gauss-Seidel algorithm [Eq. (11.5)] can be expressed as

$$u(x_1, x_2) = \frac{b_1}{a_{11}} - \frac{a_{12}}{a_{11}} x_2 \quad (11.8a)$$

and

$$v(x_1, x_2) = \frac{b_2}{a_{22}} - \frac{a_{21}}{a_{22}} x_1 \quad (11.8b)$$

The partial derivatives of these equations can be evaluated with respect to each of the unknowns as

$$\frac{\partial u}{\partial x_1} = 0 \quad \frac{\partial u}{\partial x_2} = -\frac{a_{12}}{a_{11}}$$

and

$$\frac{\partial v}{\partial x_1} = -\frac{a_{21}}{a_{22}} \frac{\partial v}{\partial x_2} = 0$$

which can be substituted into Eq. (11.7) to give

$$\left| \frac{a_{12}}{a_{11}} \right| < 1 \quad (11.9a)$$

and

$$\left| \frac{a_{21}}{a_{22}} \right| < 1 \quad (11.9b)$$

In other words, the absolute values of the slopes of Eq. (11.8) must be less than unity to ensure convergence. This is displayed graphically in Fig. 11.5. Equation (11.9) can also be reformulated as

$$|a_{11}| > |a_{12}|$$

and

$$|a_{22}| > |a_{21}|$$

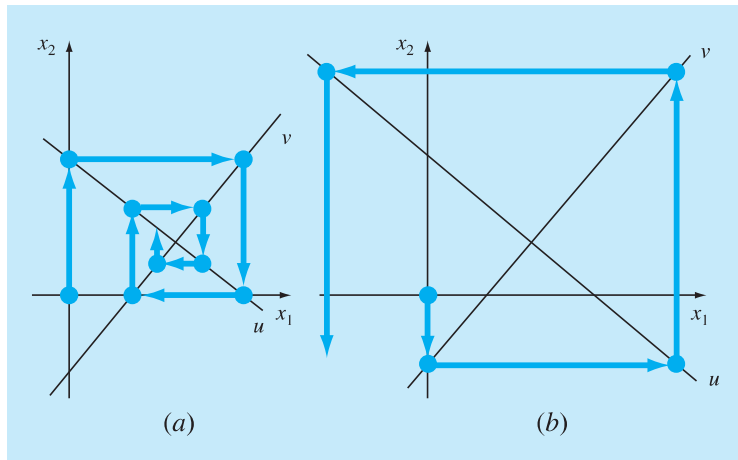
That is, the diagonal element must be greater than the off-diagonal element for each row.

The extension of the above to n equations is straightforward and can be expressed as

$$|a_{ii}| > \sum_{j=1, j \neq i}^n |a_{ij}| \quad (11.10)$$

FIGURE 11.5

Iteration cobwebs illustrating (a) convergence and (b) divergence of the Gauss-Seidel method. Notice that the same functions are plotted in both cases ($u: 11x_1 + 13x_2 = 286$; $v: 11x_1 - 9x_2 = 99$). Thus, the order in which the equations are implemented (as depicted by the direction of the first arrow from the origin) dictates whether the computation converges.



That is, the diagonal coefficient in each of the equations must be larger than the sum of the absolute values of the other coefficients in the equation. This criterion is sufficient but not necessary for convergence. That is, although the method may sometimes work if Eq. (11.10) is not met, convergence is guaranteed if the condition is satisfied. Systems where Eq. (11.10) holds are called *diagonally dominant*. Fortunately, many engineering problems of practical importance fulfill this requirement.

11.2.2 Improvement of Convergence Using Relaxation

Relaxation represents a slight modification of the Gauss-Seidel method and is designed to enhance convergence. After each new value of x is computed using Eq. (11.5), that value is modified by a weighted average of the results of the previous and the present iterations:

$$x_i^{\text{new}} = \lambda x_i^{\text{new}} + (1 - \lambda)x_i^{\text{old}} \quad (11.11)$$

where λ is a weighting factor that is assigned a value between 0 and 2.

If $\lambda = 1$, $(1 - \lambda)$ is equal to 0 and the result is unmodified. However, if λ is set at a value between 0 and 1, the result is a weighted average of the present and the previous results. This type of modification is called *underrelaxation*. It is typically employed to make a nonconvergent system converge or to hasten convergence by dampening out oscillations.

For values of λ from 1 to 2, extra weight is placed on the present value. In this instance, there is an implicit assumption that the new value is moving in the correct direction toward the true solution but at too slow a rate. Thus, the added weight of λ is intended to improve the estimate by pushing it closer to the truth. Hence, this type of modification, which is called *overrelaxation*, is designed to accelerate the convergence of an already convergent system. The approach is also called *successive* or *simultaneous overrelaxation*, or *SOR*.

The choice of a proper value for λ is highly problem-specific and is often determined empirically. For a single solution of a set of equations it is often unnecessary. However, if the system under study is to be solved repeatedly, the efficiency introduced by a wise choice of λ can be extremely important. Good examples are the very large systems of partial differential equations that often arise when modeling continuous variations of variables (recall the distributed system depicted in Fig. PT3.1*b*). We will return to this topic in Part Eight.

11.2.3 Algorithm for Gauss-Seidel

An algorithm for the Gauss-Seidel method, with relaxation, is depicted in Fig. 11.6. Note that this algorithm is not guaranteed to converge if the equations are not input in a diagonally dominant form.

The pseudocode has two features that bear mentioning. First, there is an initial set of nested loops to divide each equation by its diagonal element. This reduces the total number of operations in the algorithm. Second, notice that the error check is designated by a variable called *sentinel*. If any of the equations has an approximate error greater than the stopping criterion (e_s), then the iterations are allowed to continue. The use of the sentinel

```

SUBROUTINE Gseid (a,b,n,x,imax,es,lambd)
  DOFOR i = 1,n
    dummy = ai,i
    DOFOR j = 1,n
      ai,j = ai,j/dummy
    END DO
    bi = bi/dummy
  END DO
  DOFOR i = 1, n
    sum = bi
    DOFOR j = 1, n
      IF i ≠ j THEN sum = sum - ai,j*xj
    END DO
    xi=sum
  END DO
  iter=1
  DO
    sentinel = 1
    DOFOR i = 1,n
      old = xi
      sum = bi
      DOFOR j = 1,n
        IF i ≠ j THEN sum = sum - ai,j*xj
      END DO
      xi = lambda*sum +(1.-lambda)*old
      IF sentinel = 1 AND xi ≠ 0. THEN
        ea = ABS((xi - old)/xi)*100.
        IF ea > es THEN sentinel = 0
      END IF
    END DO
    iter = iter + 1
    IF sentinel = 1 OR (iter ≥ imax) EXIT
  END DO
END Gseid

```

FIGURE 11.6

Pseudocode for Gauss-Seidel with relaxation.

allows us to circumvent unnecessary calculations of error estimates once one of the equations exceeds the criterion.

11.2.4 Problem Contexts for the Gauss-Seidel Method

Aside from circumventing the round-off dilemma, the Gauss-Seidel technique has a number of other advantages that make it particularly attractive in the context of certain engineering problems. For example, when the matrix in question is very large and very sparse (that is, most of the elements are zero), elimination methods waste large amounts of computer memory by storing zeros.

At the beginning of this chapter, we saw how this shortcoming could be circumvented if the coefficient matrix is banded. For nonbanded systems, there is usually no simple way to avoid large memory requirements when using elimination methods. Because all computers have a finite amount of memory, this inefficiency can place a constraint on the size of systems for which elimination methods are practical.

Although a general algorithm such as the one in Fig. 11.6 is prone to the same constraint, the structure of the Gauss-Seidel equations [Eq. (11.5)] permits concise programs to be developed for specific systems. Because only nonzero coefficients need be included in Eq. (11.5), large savings of computer memory are possible. Although this entails more up-front investment in software development, the long-term advantages are substantial when dealing with large systems for which many simulations are to be performed. Both lumped- and distributed-variable systems can result in large, sparse matrices for which the Gauss-Seidel method has utility.

11.3 LINEAR ALGEBRAIC EQUATIONS WITH SOFTWARE PACKAGES

Software packages have great capabilities for solving systems of linear algebraic equations. Before describing these tools, we should mention that the approaches described in Chap. 7 for solving nonlinear systems can be applied to linear systems. However, in this section, we will focus on the approaches that are expressly designed for linear equations.

11.3.1 Excel

There are two ways to solve linear algebraic equations with Excel: (1) using the Solver tool or (2) using matrix inversion and multiplication functions.

Recall that one way to determine the solution of linear algebraic equations is

$$\{X\} = [A]^{-1}\{B\} \quad (11.12)$$

Excel has built-in functions for both matrix inversion and multiplication that can be used to implement this formula.

EXAMPLE 11.4

Using Excel to Solve Linear Systems

Problem Statement. Recall that in Chap. 10 we introduced the Hilbert matrix. The following system is based on the Hilbert matrix. Note that it is scaled, as was done previously in Example 10.3, so that the maximum coefficient in each row is unity.

$$\begin{bmatrix} 1 & 1/2 & 1/3 \\ 1 & 2/3 & 1/2 \\ 1 & 3/4 & 3/5 \end{bmatrix} \begin{Bmatrix} x_1 \\ x_2 \\ x_3 \end{Bmatrix} = \begin{Bmatrix} 1.833333 \\ 2.166667 \\ 2.35 \end{Bmatrix}$$

The solution to this system is $\{X\}^T = [1 \ 1 \ 1]$. Use Excel to obtain this solution.

Solution. The spreadsheet to solve this problem is displayed in Fig. 11.7. First, the matrix $[A]$ and the right-hand-side constants $\{B\}$ are entered into the spreadsheet cells. Then, a set of cells of the proper dimensions (in our example 3×3) is highlighted by either clicking and dragging the mouse or by using the arrow keys while depressing the shift key. As in Fig. 11.7, we highlight the range: B5..D7.

	A	B	C	D	E	F
1		1	0.5	0.33333333		1.83333333333333
2	[A] =	1	0.66666667	0.5	{B} =	2.16666666666667
3		1	0.75	0.6		2.35000000000000
4						
5		9	-18	10		0.99999999999992
6	[A] ⁻¹ =	-36	96	-60	{X} =	1.00000000000043
7		30	-90	60		0.99999999999960


=MINVERSE(B1:D3)

=MMULT(B5:D7,F1:F3)

FIGURE 11.7

Next, a formula invoking the matrix inverse function is entered,

```
=minverse(B1..D3)
```

Note that the argument is the range holding the elements of $[A]$. The *Ctrl* and *Shift* keys are held down while the *Enter* key is depressed. The resulting inverse of $[A]$ will be calculated by Excel and displayed in the range B5..D7 as shown in Fig. 11.7.

A similar approach is used to multiply the inverse by the right-hand-side vector. For this case, the range from F5..F7 is highlighted and the following formula is entered

```
=mmult(B5..D7,F1..F3)
```

where the first range is the first matrix to be multiplied, $[A]^{-1}$, and the second range is the second matrix to be multiplied, $\{B\}$. By again using the *Ctrl-Shift-Enter* combination, the solution $\{X\}$ will be calculated by Excel and displayed in the range F5..F7, as shown in Fig. 11.7. As can be seen, the correct answer results.

Notice that we deliberately reformatted the results in Example 11.4 to show 15 digits. We did this because Excel uses double-precision to store numerical values. Thus, we see that round-off error occurs in the last two digits. This implies a condition number on the order of 100, which agrees with the result of 451.2 originally calculated in Example 10.3. Excel does not have the capability to calculate a condition number. In most cases, particularly because it employs double-precision numbers, this does not represent a problem. However, for cases where you suspect that the system is ill-conditioned, determination of the condition number is useful. MATLAB and Mathcad software are capable of computing this quantity.

11.3.2 MATLAB

As the name implies, MATLAB (short for MATrix LABoratory) was designed to facilitate matrix manipulations. Thus, as might be expected, its capabilities in this area are

TABLE 11.1 MATLAB functions to implement matrix analysis and numerical linear algebra.

Matrix Analysis		Linear Equations	
Function	Description	Function	Description
cond	Matrix condition number	\ and /	Linear equation solution; use “help slash”
norm	Matrix or vector norm	chol	Cholesky factorization
rcond	LINPACK reciprocal condition estimator	lu	Factors from Gauss elimination
rank	Number of linearly independent rows or columns	inv	Matrix inverse
det	Determinant	qr	Orthogonal-triangular decomposition
trace	Sum of diagonal elements	qdelete	Delete a column from the QR factorization
null	Null space	qinsert	Insert a column in the QR factorization
orth	Orthogonalization	nls	Nonnegative least squares
rref	Reduced row echelon form	pinv	Pseudoinverse
		lsqov	Least squares in the presence of known covariance

excellent. Some of the key MATLAB functions related to matrix operations are listed in Table 11.1. The following example illustrates a few of these capabilities.

EXAMPLE 11.5**Using MATLAB to Manipulate Linear Algebraic Equations**

Problem Statement. Explore how MATLAB can be employed to solve and analyze linear algebraic equations. Use the same system as in Example 11.4.

Solution. First, we can enter the $[A]$ matrix and the $\{B\}$ vector,

```
>> A = [ 1  1/2  1/3 ; 1  2/3  1/2 ; 1  3/4  3/5 ]
```

```
A =
    1.0000    0.5000    0.3333
    1.0000    0.6667    0.5000
    1.0000    0.7500    0.6000
```

```
>> B = [1+1/2+1/3; 1+2/3+2/4; 1+3/4+3/5]
```

```
B =
    1.8333
    2.1667
    2.3500
```

Next, we can determine the condition number for $[A]$, as in

```
>> cond(A)

ans =
    366.3503
```

This result is based on the spectral, or $\|A\|_2$, norm discussed in Box 10.2. Note that it is of the same order of magnitude as the condition number = 451.2 based on the row-sum norm in Example 10.3. Both results imply that between two and three digits of precision could be lost.

Now we can solve the system of equations in two different ways. The most direct and efficient way is to employ backslash, or “left division”:

```
>> X=A\B

X =
    1.0000
    1.0000
    1.0000
```

For cases such as ours, MATLAB uses Gauss elimination to solve such systems.

As an alternative, we can implement Eq. (PT3.6) directly, as in

```
>> X=inv(A)*B

X =
    1.0000
    1.0000
    1.0000
```

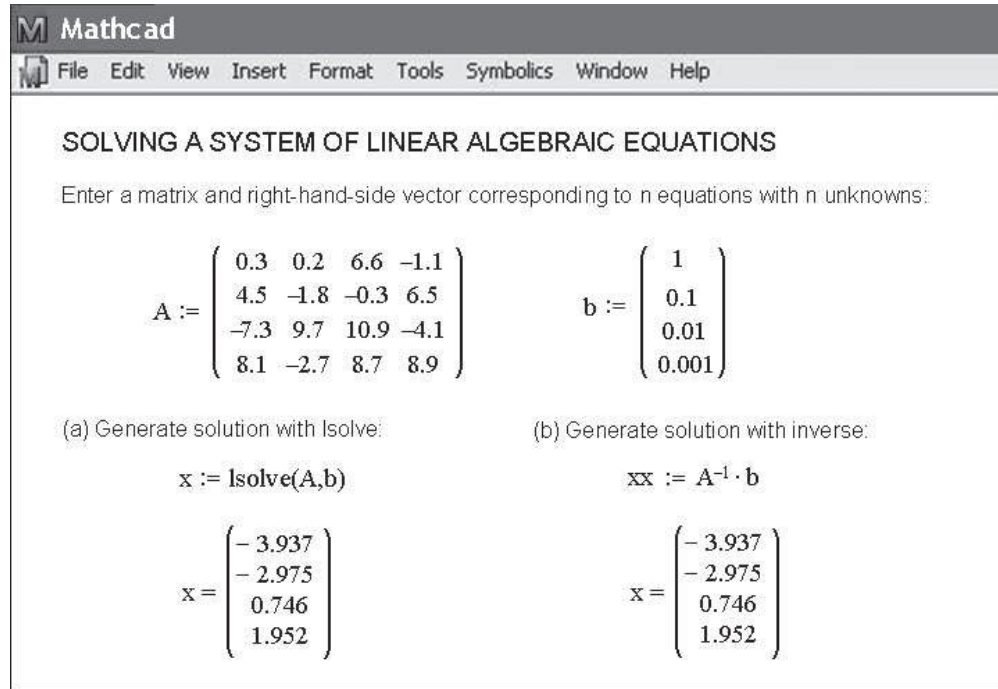
This approach actually determines the matrix inverse first and then performs the matrix multiplication. Hence, it is more time consuming than using the backslash approach.

11.3.3 Mathcad

Mathcad contains many special functions that manipulate vectors and matrices. These include common operations such as the dot product, matrix transpose, matrix addition, and matrix multiplication. In addition, it allows calculation of the matrix inverse, determinant, trace, various types of norms, and condition numbers based on different norms. It also has several functions that decompose matrices.

Systems of linear equations can be solved in two ways by Mathcad. First, it is possible to use matrix inversion and subsequent multiplication by the right-hand-side as discussed in Chap. 10. In addition, Mathcad has a special function called **lsolve(A,b)** that is specifically designed to solve linear equations. You can use other built-in functions to evaluate the condition of A to determine if A is nearly singular and thus possibly subject to round-off errors.

As an example, let's use **lsolve** to solve a system of linear equations. As shown in Fig. 11.8, the first step is to enter the coefficients of the A matrix using the definition symbol and the Insert/Matrix pull down menu. This gives a box that allows you to

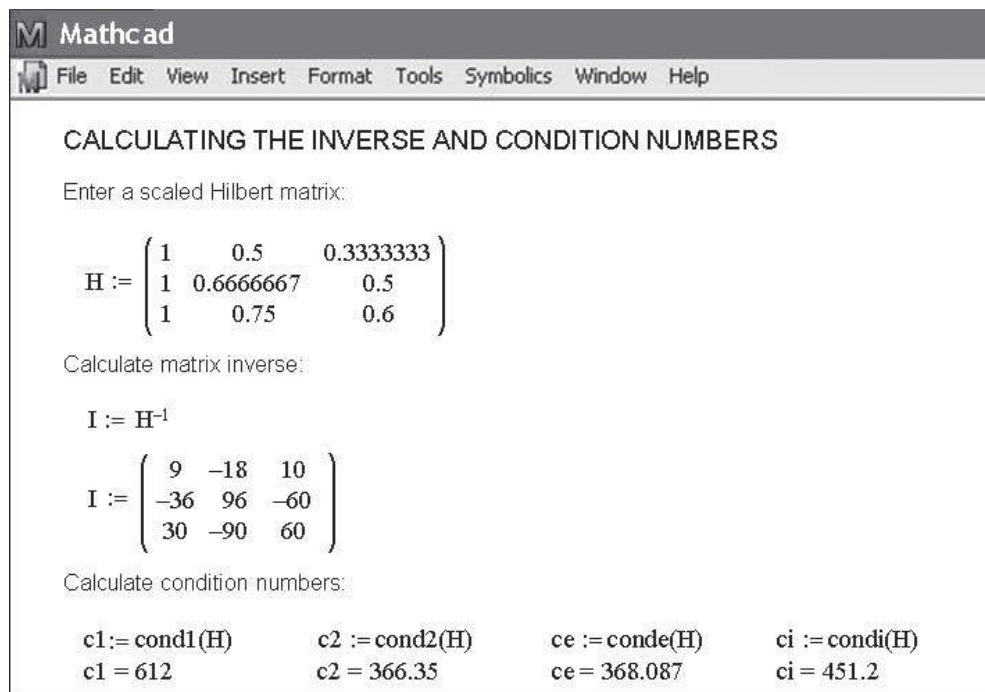
**FIGURE 11.8**

Mathcad screen to solve a system of linear algebraic equations.

specify the dimensions of the matrix. For our case, we will select a dimension of 4×4 , and Mathcad places a blank 4-by-4-size matrix on screen. Now, simply click the appropriate cell location and enter values. Repeat similar operations to create the right-hand-side b vector. Now the vector x is defined as **Isolve**(A, b) and the value of x is displayed with the equal sign.

We can also solve the same system using the matrix inverse. The inverse can be simply computed by merely raising A to the exponent -1 . The result is shown on the right side of Fig. 11.8. The solution is then generated as the product of the inverse times b .

Next, let's use Mathcad to find the inverse and the condition number of the Hilbert matrix. As in Fig. 11.9, the scaled matrix can be entered using the definition symbol and the Insert/Matrix pull down menu. The inverse can again be computed by simply raising H to the exponent -1 . The result is shown in Fig. 11.9. We can then use some other Mathcad functions to determine condition numbers by using the definition symbol to define variables $c1$, $c2$, ce , and ci as the condition number based on the column-sum (**cond1**), spectral (**cond2**), the Euclidean (**conde**), and the row-sum (**condi**) norms, respectively. The resulting values are shown at the bottom of Fig. 11.9. As expected, the spectral norm provides the smallest measure of magnitude.

**FIGURE 11.9**

Mathcad screen to determine the matrix inverse and condition numbers of a scaled 3×3 Hilbert matrix.

PROBLEMS

11.1 Perform the same calculations as in (a) Example 11.1, and (b) Example 11.3, but for the tridiagonal system,

$$\begin{bmatrix} 0.8 & -0.4 & \\ -0.4 & 0.8 & -0.4 \\ & -0.4 & 0.8 \end{bmatrix} \begin{Bmatrix} x_1 \\ x_2 \\ x_3 \end{Bmatrix} = \begin{Bmatrix} 41 \\ 25 \\ 105 \end{Bmatrix}$$

11.2 Determine the matrix inverse for Example 11.1 based on the LU decomposition and unit vectors.

11.3 The following tridiagonal system must be solved as part of a larger algorithm (Crank-Nicolson) for solving partial differential equations:

$$\begin{bmatrix} 2.01475 & -0.020875 & & \\ -0.020875 & 2.01475 & -0.020875 & \\ & -0.020875 & 2.01475 & -0.020875 \\ & & -0.020875 & 2.01475 \end{bmatrix} \times \begin{Bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{Bmatrix} = \begin{Bmatrix} 4.175 \\ 0 \\ 0 \\ 2.0875 \end{Bmatrix}$$

Use the Thomas algorithm to obtain a solution.

11.4 Confirm the validity of the Cholesky decomposition of Example 11.2 by substituting the results into Eq. (11.2) to see if the product of $[L]$ and $[L]^T$ yields $[A]$.